Compiler Optimizations and Hardware Analysis in the PyTorch-to-Calyx Pipeline through MLIR and CIRCT

ACM SIGPLAN Long-Term Mentorship Program (SIGPLAN-M) Technical Report

ABHILEKH GAUTAM

We lower a PyTorch Convolutional Neural Network (CNN) through the CIRCT compiler pipeline (Linalg → Affine → SCF → Calyx IR) and analyze the generated Calyx hardware. We examine the performance bottlenecks introduced during multidimensional memory flattening, implementing custom loop strength reduction and algebraic canonicalization passes to eliminate costly address calculation multipliers and dividers. To further optimize the generated hardware, we apply loop unrolling and memory banking, reducing cycle counts from 26 million to 18 million cycles. Furthermore, leveraging the Calyx performance profiler, we identify and promote 53 deterministic groups to static timing groups, eliminating dynamic handshake overhead and cutting cycle count further to approximately 6 million (a 67% cycle count reduction and 77% overall). Finally, we expose a structural limitation in the CIRCT `scf-to-calyx` pass where loop-carried state is forced into Block RAM (`seq_mem`) via `scf.while`, sacrificing compile-time static scheduling.

Contents

Abstract 1
Contents 1
1 Introduction & Baseline Pipeline 2
1.1 Benchmark Model 2
2 Memory Flattening 3
2.1 Why Memory Flattening Introduces Stride Arithmetic 3
3 Compiler Transformations for Hardware-Oriented Address Optimization 3
3.1 Loop Strength Reduction 4
3.2 Loop Coalescing & Algebraic Identity Elimination (ReLU) 5
3.3 Loop Unrolling and Memory Banking 6
4 Code Generation & Control Path Analysis 6
4.1 Calyx Control Flow Sequence 6
4.2 Calyx Datapath Group Mapping 7
4.3 Profiler-Guided Static Group Promotion 7
5 Limitations of Loop-Carried State in CIRCT 8
6 Quantitative Evaluation of IR Transformations 9
6.1 MLIR Operator Reduction 9
6.2 Generated Calyx Hardware Resource 9
6.3 Dynamic Execution Cycle Reduction 10
7 Conclusion 10
References 10

This technical report was conducted under the mentorship of **Dr. Kevin Laeufer** (Cornell University) as part of the ACM SIGPLAN Long-Term Mentorship Program (SIGPLAN-M).
Author's Contact Information: Abhilekh Gautam, abhilekhgautam119@gmail.com.

1 Introduction & Baseline Pipeline

MLIR [3] and CIRCT [1] provide a multi-level compilation framework that progressively lowers high-level programs through a series of intermediate representations before generating hardware. We study the compilation of a PyTorch Convolutional Neural Network (CNN) through this pipeline, examining the transformations at each dialect level and implementing targeted optimizations to improve the generated hardware.

We analyze the compilation across multiple abstraction layers, tracking how high-level machine learning operations are transformed and how these transformations impact the Calyx [5] generated hardware.

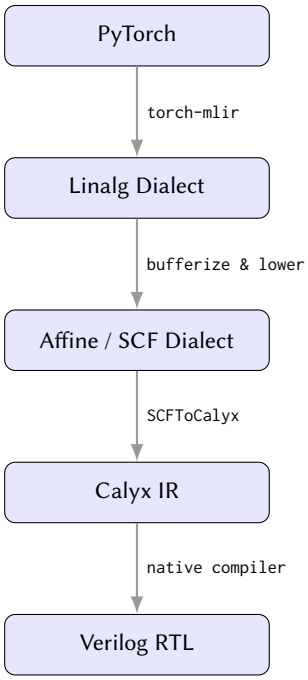


Fig. 1. The compiler dialect lowering stages, mapping high-level abstractions to hardware descriptions.

1.1 Benchmark Model

The convolutional neural network workload evaluated in this study consists of a convolutional layer (5×5 kernel, 3 input channels, 8 output channels), a ReLU activation, a 2×3 max-pooling layer, and a linear classifier.

At the entry point of our compiler pipeline, the model's forward pass is lowered to the following `linalg` dialect, operating on tensors:

```
%1 = linalg.conv_2d_nchw_fchw ins(%arg0, %weight) outs(%init_conv) -> tensor<1x8x76x56xf32>
%2 = linalg.generic ins(%1) outs(%init_relu) { ... ReLU select ... } -> tensor<1x8x76x56xf32>
%3 = linalg.pooling_nchw_max ins(%2) outs(%init_pool) -> tensor<1x8x38x18xf32>
%4 = tensor.collapse_shape %3 [[0], [1, 2, 3]] : tensor<1x8x38x18xf32> into tensor<1x5472xf32>
%5 = linalg.matmul ins(%4, %fc_weight) outs(%init_fc) -> tensor<1x2xf32>
```

2 Memory Flattening

The model operates on multi-dimensional tensors. However, the current MLIR to Calyx lowering pipeline requires these memories to be flattened into one-dimensional `memref` before hardware generation [6]. This introduces additional stride arithmetic to translate multi-dimensional indices into linear memory addresses. The resulting address calculations become a significant source of arithmetic overhead in the generated IR.

2.1 Why Memory Flattening Introduces Stride Arithmetic

When a compiler lowers an N-dimensional `memref` to flat `memref`, it must calculate the flattened index. For a tensor of shape $D_0 \times D_1 \times \cdots \times D_{N-1}$ accessed with coordinate indices $(i_0, i_1, \ldots, i_{N-1})$, the flattened index is computed by:

$$\text{Index} = \sum_{j=0}^{N-1} \left(i_j \times \text{Stride}_j \right) \quad \text{where} \quad \text{Stride}_j = \prod_{k=j+1}^{N-1} D_k$$

This calculation introduces $N - 1$ multiplication operations per memory access. In our convolution layer, flattening the 4D input and output buffers introduces a significant arithmetic overhead. Tracing this coordinate translation (4D to 1D) down to the Structured Control Flow (`scf`) dialect reveals the exact calculations injected by the `flatten-memref` pass for the input and output buffer loads:

```
scf.for %arg3 = %c0 to %c8 step %c1 { // Channels Out
  scf.for %arg4 = %c0 to %c76 step %c1 { // Height Out
    scf.for %arg5 = %c0 to %c56 step %c1 { // Width Out
      // ... Inner reduction loops (arg6, arg7, arg8) omitted ...
      // Input Index Calculation (4D to 1D)
      %13 = arith.muli %12, %c60 : index
      %14 = arith.addi %13, %17 : index
      %15 = memref.load %arg0[%14] : memref<14400xf32>

      // Output Accumulator Index Calculation (4D to 1D)
      %24 = arith.muli %23, %c56 : index
      %25 = arith.addi %24, %arg5 : index
      %26 = memref.load %arg2[%25] : memref<34048xf32>
    }
  }
}
```

These index multipliers (`arith.muli`) are nested deep within our loop structure. On an FPGA, synthesizing these multiplications directly into hardware logic creates a severe bottleneck. During synthesis, these multiplications are typically mapped either to dedicated DSP blocks (which are a highly limited physical resource) or LUT-based multiplier implementations that drastically reduce the maximum clock frequency ($F_{\max}$) of the chip.

3 Compiler Transformations for Hardware-Oriented Address Optimization

We implement two custom passes: Loop Strength reduction (Section 3.1) targets the $i \times C$ pattern and algebraic identity elimination (Section 3.2) targets the

$$(X/C)\rfloor \times C + (X\%C) \implies X$$

pattern.

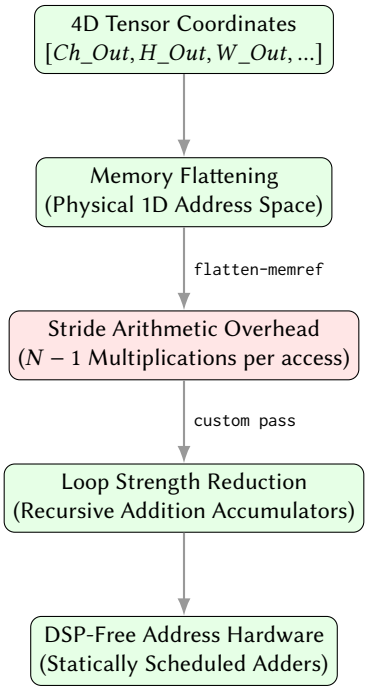


Fig. 2. The memory flattening and loop optimization pipeline, converting multi-dimensional indices to hardware-efficient addition-based address datapaths.

3.1 Loop Strength Reduction

We implemented a custom loop strength reduction pass [2] in C++ to optimize computation involving loop induction variables and loop invariants. The pass targets multiplication within a loop where one term is a monotonically increasing induction variable (i) and the other is a loop-invariant stride constant (C). Rather than computing $i \times C$ multiplication blocks on each iteration, our pass performs a rewrite that replaces the multiplication with a low-cost addition operation.

3.1.1 Before Transformation. Prior to running our pass, the index arithmetic relies on standard multiplication nested inside loop boundaries:

```
scf.for %arg1 = %c0 to %c8 step %c1 {
  // Stride arithmetic requiring hardware multipliers
  %idx = arith.muli %arg1, %c60 : index
  %val = memref.load %arg0[%idx] : memref<14400xf32>
}
```

3.1.2 After Transformation. Our pass rewrites this loop block, allocating a local loop-invariant `memref` initialized to 0, and incrementing it by the stride constant on each iteration. This shifts the computational cost from multiplication to addition:

```
%accumulated_sum = memref.alloca() : memref<index>
memref.store %c0, %accumulated_sum[] : memref<index>
```

```
scf.for %arg1 = %c0 to %c8 step %c1 {
  %acc_val = memref.load %accumulated_sum[] : memref<index>
  %val = memref.load %arg0[%acc_val] : memref<14400xf32>

  // Accumulate the index via low-cost addition
  %next = arith.addi %acc_val, %c60
  memref.store %next, %accumulated_sum[] : memref<index>
}
```

This pattern is recursively applied across all loop nest in the pipeline, successfully eliminating costly multiplications.

3.2 Loop Coalescing & Algebraic Identity Elimination (ReLU)

The element-wise ReLU activation layer introduces the same index-arithmetic cost as the convolution kernel after flattening.

```
affine.for %arg1 = 0 to 1 {
  affine.for %arg2 = 0 to 8 {
    affine.for %arg3 = 0 to 76 {
      affine.for %arg4 = 0 to 56 {
        %4 = affine.load %alloc[%arg1, %arg2, %arg3, %arg4] : memref<1x8x76x56xf32>
        %5 = arith.cmpf ugt, %4, %cst_0 : f32
        %6 = arith.select %5, %4, %cst_0 : f32
        affine.store %6, %alloc[%arg1, %arg2, %arg3, %arg4] : memref<1x8x76x56xf32>
      }
    }
  }
}
```

When compiled to SCF and flattened, this introduces an extensive sequence of address calculation multiplications. ReLU contains no intrinsic multiplications, so the index-arithmetic cost is a pure overhead.

To optimize this, we run the `affine-loop-coalescing` pass, which collapses the multidimensional loop nest into a single, unified 1D iteration space [4]:

```
scf.for %arg1 = %c0 to %c34048 step %c1 {
  %4 = arith.remsi %arg1, %c56 : index
  %5 = arith.divsi %arg1, %c56 : index
  %6 = arith.remsi %5, %c76 : index
  %7 = arith.divsi %5, %c76 : index
  %8 = arith.muli %7, %c76 : index
  %9 = arith.addi %8, %6 : index
  %10 = arith.muli %9, %c56 : index
  %11 = arith.addi %10, %4 : index
  %12 = memref.load %alloc[%11] : memref<34048xf32>
  %13 = arith.cmpf ugt, %12, %cst_0 : f32
  %14 = arith.select %13, %12, %cst_0 : f32
  memref.store %14, %alloc[%11] : memref<34048xf32>
}
```

While loop coalescing simplifies the loop structure, it introduces costly signed division (`divsi`) and signed modulo (`remsi`) operators in the index-regeneration logic. On FPGAs, signed division and modulo are not native operations. They are expensive in both area and critical-path delay.

To eliminate this overhead, we implemented a custom algebraic simplification pass. The pass performs a pattern-match over the index computation graph, looking for the canonical division-modulo reconstruction identity:

$$(X/C)\rfloor \times C + (X\%C) \implies X$$

Our pass matches this exact pattern and replaces the entire calculation of `%11` directly with the loop induction variable `%arg1`, rendering the division, modulo, and multiplication operations redundant. This leaves us with an optimized, multiplication-free ReLU loop:

```
scf.for %arg1 = %c0 to %c34048 step %c1 {
  %12 = memref.load %alloc[%arg1] : memref<34048xf32>
  %13 = arith.cmpf ugt, %12, %cst_0 : f32
  %14 = arith.select %13, %12, %cst_0 : f32
  memref.store %14, %alloc[ %arg1 ] : memref<34048xf32>
}
```

This eliminates the multiplier and divider resource overhead in the synthesized ReLU datapath.

3.3 Loop Unrolling and Memory Banking

Following the multi-level optimization methodology outlined by Xie et al. [6], we apply aggressive loop unrolling coupled with memory banking to maximize data parallelism.

To expose maximum data parallelism, we fully unroll the innermost reduction loops corresponding to the input channels and convolution kernel dimensions ($3 \times 5 \times 5 = 75$ total unrolled iterations). However, full unrolling multiplies concurrent memory access requests, creating an immediate read/write port bottleneck on 1D `memref` buffers.

To support 75 concurrent memory loads without stalling the pipeline, we partition and bank the memory buffers by a factor of 75 into separate physical `calyx.seq_mem` blocks. This banking transformation allows all 75 unrolled processing elements to access their required data in parallel, successfully reducing the cycle count from 26 million to 18 million cycles.

However, this optimization introduces a severe area trade-off: partitioning the memory by a factor of 75 results in excessive resource utilization that may become unfeasible on area-constrained FPGA devices.

4 Code Generation & Control Path Analysis

Lowering our optimized MLIR representation down to Calyx compiles the nested `scf.for` structures into Calyx groups.

4.1 Calyx Control Flow Sequence

The generated Calyx control structure translates the multi-dimensional loop nests using sequential and repeating control blocks. To illustrate this transition clearly, the nested control sequence is outlined below, highlighting the hierarchy of static `repeat` blocks generated by the compiler:

```
repeat 8 { // Outer loop %arg2
  seq {
    repeat 76 { // Loop %arg3
      seq {
        repeat 56 { // Loop %arg4
          seq {
            repeat 3 { // Loop %arg5
              seq {
                repeat 5 { // Loop %arg6
```

```
                seq {
                  repeat 5 { // Innermost loop %arg7 (Kernel calculations)
                    seq { bb0_42; bb0_44; bb0_45; bb0_46; ... bb0_50; }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

This control-path sequence mirrors our optimized MLIR loop hierarchy using the static `repeat` operator. The innermost loop contains sequential datapath blocks (such as `bb0_42` to `bb0_50`) representing our active convolution kernel.

4.2 Calyx Datapath Group Mapping

Examining a specific, compiled Calyx wire group shows how MLIR arithmetic is mapped to hardware components. We isolate group `bb0_42`, which is responsible for calculating the flattened coordinate index for our input memory (`arg_mem_0`):

```
group bb0_42 {
    std_slice_19.in = std_add_9.out;

    // Addition output as the index
    arg_mem_0.addr0 = std_slice_19.out;
    arg_mem_0.content_en = 1'b1;

    std_add_9.left = load_10_reg.out;
    std_add_9.right = std_add_8.out;

    std_add_8.left = for_6_induction_var_reg.out;
    std_add_8.right = for_3_induction_var_reg.out;

    bb0_42[done] = arg_mem_0.done;
}
```

This group compiles index calculation directly to adders (`std_add_9` and `std_add_8`), routing the sliced result to the address port of the input memory.

4.3 Profiler-Guided Static Group Promotion

To diagnose control-path overheads, we leverage performance profiling techniques developed by Yorihiro et al. [7] for accelerator compilers. The Calyx profiler instruments the design to track execution latencies, per-group active cycles, and state machine transition delays.

Inspection of the profiler statistics revealed that 53 compiled Calyx groups were instantiated dynamically despite exhibiting strictly bounded, fixed-latency execution. These groups incurred continuous state register reads and done-signal handshaking overhead on every iteration.

By analyzing the profiler group statistics, we manually promoted these 53 groups to static groups (`static group`) to avoid control register update cycles. As a result, this profiler-guided static promotion reduces execution time from 18 million cycles to approximately 6 million cycles, achieving a nearly 67% cycle count reduction.

5 Limitations of Loop-Carried State in CIRCT

Our custom strength reduction pass successfully eliminates DSP-multiplying index calculations, but the generated Calyx IR has a bottleneck introduced by the MLIR abstraction layer.

In our initial optimization pass, the iteration accumulators were instantiated using local memory allocations (`memref.alloca`):

```
%alloca_18 = memref.alloca() : memref<index>
memref.store %17, %alloca_18[] : memref<index>
// ... inside inner loop ...
%22 = memref.load %alloca_18[] : memref<index>
%24 = arith.addi %22, %c60 : index
memref.store %24, %alloca_18[] : memref<index>
```

CIRCT's `scf-to-calyx` pass translates these local `alloca` variables into physical sequential memory components in Calyx (`seq_mem`):

```
%mem_18.addr0, ... = calyx.seq_mem @mem_18 <[1] x 32> [1] : i1, i1, i1, i1, i32, i32, i1
```

Because these variables map to synchronous, single-element block RAMs (BRAMs) rather than registers, they introduce a memory access penalty:

(1) **Synchronous Access Latency:** Accessing a `seq_mem` requires 1-cycle read latency and a 1-cycle write latency. To handle this, the Calyx control path must execute sequential read/write states, which can increase the loop's Initiation Interval (II).

To bypass these latencies, the compiler should promote these variables to registers (`std_reg`). In standard MLIR, this is achieved by passing loop variables as loop-carried state using the `iter_args` parameter of the `scf.for` operation.

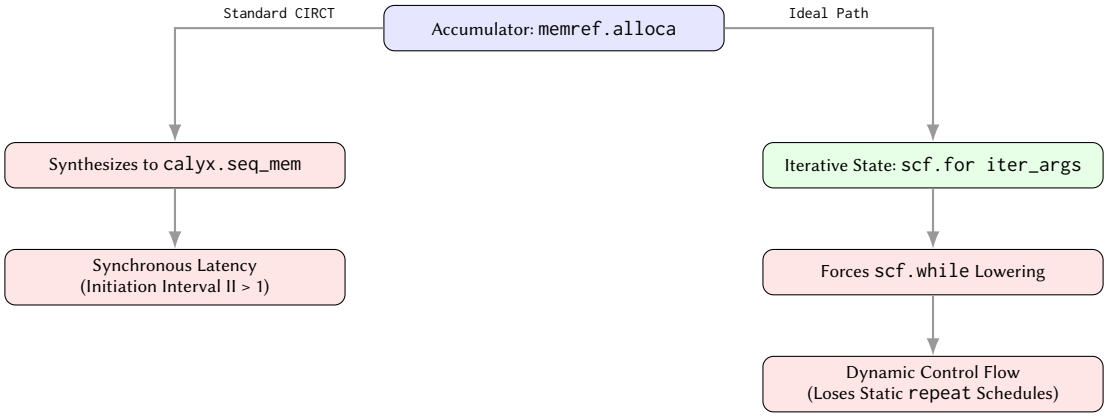


Fig. 3. Conservative memory compilation vs. dynamic control-flow in CIRCT.

The `scf-to-calyx` pass does not support `iter_args` on `scf.for` loops. When it encounters loop-carried state, the toolchain forces a lowering from `scf.for` to `scf.while`.

This lowering to `scf.while` has a hardware cost. A `while` loop has data-dependent termination conditions, meaning the loop bounds are no longer known at compile time. Because of this, the pipeline can no longer generate statically scheduled `calyx.repeat` operators. Instead, it must fall back to `calyx.while` operator which generates state-machine-driven logic.

Our analysis reveals a trade-off in the current CIRCT pipeline: the `scf-to-calyx` pass lowers loop-carried state through `scf.while` to keep the translation general. But doing so forces dynamic control flow and loses the static scheduling in Calyx. The fix is to support `iter_args` on `scf.for` in the `scf-to-calyx` pass, and to promote those `iter_args` directly to Calyx registers rather than `seq_mem`.

6 Quantitative Evaluation of IR Transformations

To measure the impact of our custom optimization passes across the compilation pipeline, we track the operator counts in the MLIR `scf` dialect and the generated hardware components in Calyx before and after running our optimizations.

6.1 MLIR Operator Reduction

Table 1 shows the changes in arithmetic operators within the `scf` dialect.

Table 1. MLIR Operation Counts (Before vs. After Custom Transformations)

MLIR Operation (SCF Dialect)	Before Passes (Baseline)	After Custom Passes (Optimized)
`arith.muli` (Multipliers)	19	4
`arith.addi` (Adders)	22	37
`memref.alloca` (Local Mem)	0	25

Our pass successfully reduced the multipliers (`arith.muli`) from 19 to 4. The increase in adders and the introduction of 25 local allocations (`memref.alloca`) represent the cost of implementing the addition-based index accumulators.

6.2 Generated Calyx Hardware Resource

Table 2 shows the resulting hardware component counts generated by the Calyx compiler.

Table 2. Synthesized Calyx Hardware Resource Estimates

Synthesized Hardware Component	Before Passes (Baseline)	After Custom Passes (Optimized)
Address Multipliers (`std_mult_pipe`)	19	4
Hardware Registers (`calyx.register`)	65	75
Synchronous SRAMs (`calyx.seq_mem`)	9	34

The hardware metrics directly reflect the IR-level changes:

(1) **Multiplier Elimination:** Eliminating 15 index-arithmetic multipliers removes the need to generate 15 multiplier (`std_mult_pipe`), saving valuable hardware resources for the actual deep learning datapath.
(2) **Memory-Latency Penalty:** Because CIRCT cannot lower loop-carried variables to registers, the 25 new logical allocations were synthesized directly into 25 physical `calyx.seq_mem` cells. This increased the total memory footprint from 9 to 34, illustrating the trade-off discussed in Section 5.

6.3 Dynamic Execution Cycle Reduction

Table 3 reports the progressive execution cycle reductions achieved by combining IR-level arithmetic simplification, loop unrolling with memory banking, and profiler-guided static group promotion.

Table 3. Dynamic Execution Cycle Progression Across Optimization Stages

Optimization Stage	**Cycle Count**	**Relative Stage Reduction**
Baseline Hardware	26,989,895	–
+ Loop Unrolling & Memory Banking [6]	18,779,724	30.4%
+ Profiler-Guided Static Promotion (53 Groups) [7]	5,978,881	68.2%
Cumulative Reduction		**77.8% overall cycle reduction**

The initial pipeline required approximately 27M execution cycles. Applying loop unrolling and banking parallelized kernel evaluation down to 18.8M cycles. Finally, using the Calyx profiler to identify and promote 53 deterministic groups to static timing components removed dynamic handshaking overhead, bringing the total latency down to under 6.0M cycles (a 68.2% improvement over the banked baseline and 77.8% overall).

7 Conclusion

This report has characterized the progressive compilation, progressive lowering, and multi-level optimizations required to map a PyTorch CNN to hardware using MLIR, CIRCT, and Calyx.

At the IR level, our custom loop strength reduction and algebraic canonicalization passes successfully eliminated costly multipliers and dividers from flattened multidimensional address calculations. At the architecture level, applying loop unrolling coupled with memory banking resolved memory port contention and parallelized execution, reducing the cycle count from 26 million to 18 million cycles. Furthermore, using Calyx profiler statistics, we identified and promoted 53 dynamic groups with fixed-latency behavior into static timing groups, cutting dynamic control handshaking overhead and slashing overall cycle count to approximately 6 million cycles (a cumulative 77.8% reduction).

Finally, our analysis exposed a fundamental structural design trade-off in CIRCT: the current inability of the `scf-to-calyx` lowering pass to promote loop-carried variables (`iter_args`) to registers without falling back to dynamic `scf.while` loops and forfeiting static loop schedules. Resolving this limitation represents a promising goal to unlock fully static, high-throughput hardware generation directly from MLIR.

Acknowledgments

I would like to express my sincere gratitude to my mentor, **Dr. Kevin Laeufer** (*Cornell University*), for their generous guidance, feedback, and discussions throughout this project. Their insights into the MLIR, CIRCT, and Calyx compiler ecosystems were instrumental in shaping this work.

I would also like to thank the organizers of the ACM SIGPLAN Long-Term Mentorship Program (SIGPLAN-M) for creating this mentorship opportunity.

References

[1] CIRCT Contributors. 2021. CIRCT. https://github.com/llvm/circt

[2] Keith D Cooper and Linda Torczon. 2023. *Engineering a compiler* (3rd ed.). Morgan Kaufmann.

[3] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2–14. doi:10.1109/CGO51591.2021.9370308

[4] LLVM Project. 2025. *MLIR Passes: affine-loop-coalescing*. https://mlir.llvm.org/docs/Passes/#-affine-loop-coalescing
[5] Rachit Nigam, Samuel Thomas, Zhijing Li, and Adrian Sampson. 2021. A compiler infrastructure for accelerator generators. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3445814.3446712
[6] Jiahan Xie, Evan Williams, and Adrian Sampson. 2025. From PyTorch to Calyx: An Open-Source Compiler Toolchain for ML Accelerators. doi:10.48550/arXiv.2512.06177
[7] Ayaka Yorihiro, Griffin Berlstein, Pedro Pontes García, Kevin Laeufer, and Adrian Sampson. 2025. Understanding Accelerator Compilers via Performance Profiling. In *arXiv preprint*. arXiv:2511.19764 [cs.PL] doi:10.48550/arXiv.2511.19764